

Forensics Git 1

 **Security Warning:** Work in an isolated environment (virtual machine or container). Never open unknown files on your primary machine.

Challenge Description

Can you find the flag in this disk image?

Hints

- How can you checkout the files of a previous commit?
-

Tools Used

- `7z` – Extract compressed files
 - `file` – Determine file type
 - `strings` – Extract printable character sequences
 - `exiftool` – Read metadata
 - `fdisk` – View partition tables
 - `dd` – Extract specific partitions
 - `mount` – Mount a filesystem as a loopback device
 - `tree` – Display directory structure
 - `git` – Examine repository history and commits
 - `grep` – Filter output
-

Complete Solution

Step 1: Download and Extract

Download the disk image named `disk.img.gz` from the challenge page.

Extract the compressed disk image:

```
7z x disk.img.gz
```

This decompresses the gzipped file and creates the raw disk image `disk.img`.

Step 2: Basic File Inspection

Verify the file type and understand its structure:

```
file disk.img
```

Expected output:

```
disk.img: DOS/MBR boot sector; partition 1 : ID=0x83, active, start-CHS (0x2,0,33),
end-CHS (0x263,8,56), startsector 2048, 614400 sectors; partition 2 : ID=0x82, start-
CHS (0x263,8,57), end-CHS (0x3ff,15,63), startsector 616448, 524288 sectors; partition
3 : ID=0x83, start-CHS (0x3ff,15,63), end-CHS (0x3ff,15,63), startsector 1140736,
956416 sectors
```

The output shows a **DOS/MBR boot sector** with **three partitions**:

- Partition 1: Linux (ID=0x83) – Boot partition
 - Partition 2: Linux swap (ID=0x82)
 - Partition 3: Linux (ID=0x83) – Data partition
-

Step 3: Search for Visible Text

Disk images often contain strings embedded directly in the data. Extract all readable character sequences and search for the flag:

```
strings disk.img | grep -i "picoCTF"
```

Output: (no results)

The flag is not directly visible. We need to dig deeper.

Step 4: Inspect Metadata

Examine the disk image metadata:

```
exiftool disk.img
```

Expected output:

```
ExifTool Version Number      : 13.50
File Name                    : disk.img
Directory                   : .
File Size                    : 1074 MB
File Modification Date/Time  : 2025:11:19 11:25:49+02:00
File Access Date/Time       : 2026:05:14 17:20:05+02:00
File Inode Change Date/Time  : 2026:05:14 13:40:56+02:00
File Permissions             : -rw-r--r--
Error                        : Unknown file type
```

Nothing suspicious here.

Step 5: View Partitions

Use `fdisk` to see the partition structure in detail:

```
fdisk -l disk.img
```

Expected output:

```
Disk disk.img: 1 GiB, 1073741824 bytes, 2097152 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x610b63c2
```

Device	Boot	Start	End	Sectors	Size	Id	Type
disk.img1	*	2048	616447	614400	300M	83	Linux
disk.img2		616448	1140735	524288	256M	82	Linux swap / Solaris
disk.img3		1140736	2097151	956416	467M	83	Linux

The third partition (disk.img3) is the largest and most likely to contain user data.

Step 6: Extract the Linux Partition

We will extract the third partition using `dd` :

```
dd if=disk.img of=linux_partition.dd bs=512 skip=1140736 count=956416 status=progress
```

Output:

```
457814528 bytes (458 MB, 437 MiB) copied, 9 s, 50.9 MB/s
956416+0 records in
956416+0 records out
489684992 bytes (490 MB, 467 MiB) copied, 10.7619 s, 45.5 MB/s
```

Step 7: Analyze the Extracted Partition

Check the type of the extracted partition:

```
file linux_partition.dd
```

Output:

```
linux_partition.dd: Linux rev 1.0 ext4 filesystem data, UUID=7a00e9da-98f8-4f0f-b257-95edf422d902 (extents) (64bit) (large files) (huge files)
```

Excellent! It's an **ext4 filesystem**.

Step 8: Mount the Extracted Partition

Create a mount point and mount the partition as a loopback device:

```
sudo mkdir /mnt/git1
sudo mount -o loop linux_partition.dd /mnt/git1
```

Note: The `-o loop` option allows Linux to treat a regular file as a block device.

Step 9: Explore the Mounted Image

List the contents of the root directory:

```
ls -alh /mnt/git1
```

Output:

```
total 45K
drwxr-xr-x 22 root root 1.0K Nov 19 10:38 .
drwxr-xr-x 13 root root 4.0K May 14 17:22 ..
drwxr-xr-x  2 root root 4.0K Nov 19 10:39 bin
drwxr-xr-x  2 root root 1.0K Nov 11  2025 boot
drwxr-xr-x  2 root root 1.0K Nov 11  2025 dev
drwxr-xr-x 30 root root 3.0K Nov 19 10:39 etc
drwxr-xr-x  3 root root 1.0K Nov 11  2025 home
...
```

Let's look inside the `home` directory for any interesting files (like a `.git` repository):

```
tree /mnt/git1/home -alh
```

Output:

```
[1.0K] /mnt/git1/home
├─ [1.0K] ctf-player
│   └─ [1.0K] Code
│       └─ [1.0K] secrets
│           └─ [1.0K] .git
│               ├── [ 62] COMMIT_EDITMSG
│               ├── [ 23] HEAD
│               ├── [1.0K] branches
│               ├── [ 92] config
│               ├── [ 73] description
│               ├── [1.0K] hooks
│               │   ├── [ 478] applypatch-msg.sample
│               │   └─ [ 896] commit-msg.sample
│               └─ ...
└─ ...
```

Key finding: A `.git` repository exists at `/home/ctf-player/Code/secrets/.git` !

Step 10: Examine the Git Repository History

Navigate to the repository:

```
cd /mnt/git1/home/ctf-player/Code/secrets/.git
```

First, check the commit history:

```
git log --oneline
```

Output:

```
5fb8194 (HEAD -> master) Remove flag
177789a Add flag
```

There are two commits:

- `5fb8194` – “Remove flag” (current commit)
- `177789a` – “Add flag” (previous commit)

The hint suggests we need to checkout a **previous commit** to recover the deleted flag.

Step 11: Examine the Previous Commit

Let’s view the changes in the previous commit (where the flag was added):

```
git show --pretty="" 177789a
```

Output:

```
diff --git a/flag.txt b/flag.txt
new file mode 100644
index 0000000..f150f47
--- /dev/null
+++ b/flag.txt
@@ -0,0 +1 @@
```

```
+picoCTF{<REDACTED>}
\ No newline at end of file
```

Key finding: The flag was added in commit [177789a](#) and then removed in [5fb8194](#) .

Alternative Git Commands for Forensic Analysis

Here are useful Git commands for exploring repositories and recovering deleted content:

Command	Purpose
<code>git log --oneline</code>	Show commits in a compact format
<code>git log --stat</code>	Show commit statistics (files changed)
<code>git log -p</code>	Show full diff for each commit
<code>git log --grep="flag"</code>	Search commit messages for a pattern
<code>git show <commit></code>	Show details of a specific commit
<code>git show <commit>:<file></code>	Show a specific file from a commit
<code>git diff HEAD~1 HEAD</code>	Show changes between last two commits
<code>git reflog</code>	Show reference log (recover "lost" commits)
<code>git checkout <commit></code>	Switch to a previous commit (detached HEAD)
<code>git checkout <commit> -- <file></code>	Restore a specific file from a commit
<code>git branch -a</code>	List all branches (local and remote)
<code>git fsck --lost-found</code>	Recover dangling commits/blobs
<code>git grep "picoCTF"</code>	Search across all tracked files and commits
<code>git blame flag.txt</code>	Show who last modified each line
<code>git whatchanged</code>	Show log with what changed

 Final Result

```
picoCTF{<REDACTED>}
```

Note: The actual flag is redacted here. In the real challenge, `git show 177789a` will reveal the complete flag.

 Key Concepts

Concept	Description
Disk Image (DD)	A raw sector-by-sector copy of a storage device.
MBR Partition Table	A structure that divides a disk into multiple partitions.
Partition Extraction with dd	Using <code>skip</code> and <code>count</code> to isolate a specific partition.
ext4 Filesystem	The native Linux filesystem for data partitions.
Loopback Mount	Mounting a file as a block device using <code>mount -o loop</code> .
Git Repository	A version control directory (<code>.git</code>) storing complete project history.
Git Commit History	Every commit is preserved, including deleted files.
Recovering Deleted Files	Using <code>git show <commit></code> or <code>git checkout <commit></code> to retrieve removed content.
Forensic Git Analysis	Examining <code>.git</code> directories to recover deleted files and hidden information.

 Pro Tips

1. **Always check the Git log** – Even if a file was deleted, its history remains in the repository.

2. **Use `git reflog`** – If commits appear to be missing, the reflog tracks all reference changes and can help recover lost commits.
3. **Search across all commits** – `git grep "pattern" $(git rev-list --all)` searches every commit for a pattern.
4. **Recover dangling commits** – `git fsck --lost-found` finds commits no longer referenced by any branch or tag.
5. **When dealing with disk images**, always check for `.git` , `.svn` , `.hg` , or other version control artifacts — they are goldmines for forensic analysis.